
ISYE 6832 Project: CIR Diffusion with Tweedie-Type Score Training

1 Introduction

We study score-based generative modeling for categorical data through a simplex-aware diffusion model built from the Cox–Ingersoll–Ross (CIR) process. This setting is motivated by tasks in which each object is naturally discrete: a DNA base is one of four letters, and a Sudoku cell is one of nine digits. After one-hot encoding, each token lies on a simplex vertex rather than in unconstrained Euclidean space. Standard Gaussian diffusion is therefore not a natural geometric model for these data, while simplex-based diffusion is.

Recent work has already shown that simplex-aware diffusion is promising. CIR-based constructions provide a principled Gamma–Dirichlet route to simplex diffusion [RDD22], and Dirichlet score models have shown that these ideas can work on real tasks such as promoter design and Sudoku [AST⁺23]. The high-level gap is that respecting the simplex geometry is only part of the problem. We still need a score-learning target that is easy to simulate, stable near small diffusion times, and simple enough to train with a neural network.

In the current local benchmark stack, this gap shows up concretely in the comparison between two score-learning pipelines. The direct Dirichlet baseline uses Jacob SDE as forward noising and learns a direct score-related target through transition density of the Jacob SDE, which truncate the infinite spectral series of transition density at finite. Such truncation will lead to un-analyzable error and numerical instability. To this end, we propose the central question for this paper: *Can we achieve a framework that both has numerical stability and theoretical guarantee?*

Our project addresses this gap via a Tweedie-type formula that re-expresses the score function as a conditional expectation form. Such conditional expectation enables a geometry-aware model with an exact forward simulator, simulation-friendly labels, and a standard MSE training objective. Thus, the simulation-based sampling framework achieves both stable and provable results.

High-level contributions.

1. We use the Arras–Swan interpolation formula for CIR to obtain an exact Poisson–Gamma forward simulator and a Tweedie-type identity for the CIR score [AS17].
2. We recast score learning as posterior-count regression: the neural network learns the posterior expected latent count, and the full score is recovered analytically from that quantity.
3. We evaluate the method on two benchmark families supported by the local codebase and saved runs: Sudoku completion and the official DDSM promoter-design benchmark. The empirical picture is intentionally honest. Sudoku shows a clearer downstream win, while DNA shows a cleaner learned score but not a better end-to-end application metric.

2 Literature Review

Score-based diffusion was originally developed in unconstrained Euclidean settings, where Gaussian forward noise and reverse-time score estimation are the standard design pattern [SSDK⁺21]. That

framework has been extremely successful, but it does not directly respect the nonnegativity and sum-to-one constraints of categorical one-hot data.

Simplex-based diffusion addresses that mismatch by designing the forward process directly on the positive orthant or simplex. Richemond et al. [RDD22] show that CIR-type dynamics are a natural fit because independent nonnegative coordinates can be normalized into simplex-valued states. Avdeyev et al. [AST⁺23] demonstrate that simplex diffusion can also be practical by building the DDSM benchmark pipeline for biological sequence design and by studying structured categorical tasks such as Sudoku [AST⁺23].

What remains difficult is the score-learning target itself. In our local comparison pipeline, the baseline model uses product-Dirichlet noising and learns Dirichlet concentration parameters, whose implied weighted score is concentration minus one. This is a direct score-learning strategy. By contrast, the CIR approach exposes an additional latent-variable structure through the Poisson count in the forward transition. That latent structure is the main opening exploited in this paper.

So the literature gap we focus on is narrower than the full diffusion literature but important for this project: prior simplex work gives the right geometry, but the training target can still be analytically awkward and numerically delicate, especially near small diffusion times. Our contribution is to keep the simplex-aware geometry while changing the learned quantity. Instead of learning the score directly, we learn the posterior expected latent count and recover the score analytically afterward.

3 Our Method

We first describe the orthant-level CIR construction, then explain how it is lifted to a product of simplices, and finally show how the local implementation turns the score-learning problem into supervised regression.

Product-simplex representation. Let there be B categorical blocks, each with d categories, so the state space is the product simplex

$$(\Delta^{d-1})^B = \left\{ z = (z^{(1)}, \dots, z^{(B)}) : z^{(b)} \in \mathbb{R}_+^d, \sum_{j=1}^d z_j^{(b)} = 1 \right\}.$$

The code represents this state in flattened form, but mathematically it is more convenient to keep the block notation. We evolve an orthant-valued state

$$r = (r^{(1)}, \dots, r^{(B)}) \in (\mathbb{R}_+^d)^B$$

and map it back to the product simplex by blockwise normalization:

$$\mathcal{N}(r)_j^{(b)} = \frac{r_j^{(b)}}{\sum_{\ell=1}^d r_\ell^{(b)}}, \quad b = 1, \dots, B, \quad j = 1, \dots, d.$$

To match a desired Dirichlet reference law on each block, the local implementation starts from a block concentration vector $a \in \mathbb{R}_+^d$, a stationary block mass $M > 0$, and a mean-reversion rate $\kappa > 0$. It then sets

$$s := \frac{M}{\sum_{j=1}^d a_j}, \quad \theta_j := s a_j, \quad \sigma := \sqrt{2\kappa s}.$$

For the coordinatewise CIR process with drift parameter θ_j and volatility σ , the stationary Gamma shape is

$$\alpha_j = \frac{2\kappa\theta_j}{\sigma^2} = a_j.$$

So after blockwise normalization, the stationary reference law is exactly a product of Dirichlet distributions with concentration vector a . This is how the code aligns the orthant diffusion with simplex geometry.

Exact CIR transition. For one orthant coordinate, the forward CIR process is

$$dR_t = \kappa(\theta - R_t) dt + \sigma\sqrt{R_t} dB_t, \quad R_0 = Y. \quad (1)$$

The local implementation uses the exact Poisson–Gamma transition rather than an Euler approximation. Define

$$s_t := \frac{\sigma^2(1 - e^{-\kappa t})}{2\kappa}, \quad \mu_t(y) := \frac{2\kappa e^{-\kappa t} y}{\sigma^2(1 - e^{-\kappa t})}.$$

Then the transition can be sampled by

$$K_t \mid Y = y \sim \text{Poisson}(\mu_t(y)), \quad R_t \mid (K_t, Y = y) \sim \text{Gamma}(\alpha + K_t, s_t),$$

where $\alpha = 2\kappa\theta/\sigma^2$ is the stationary Gamma shape. This is the exact Poisson–Gamma mechanism used by the local implementation. It produces training pairs (R_t, K_t) at arbitrary diffusion times without discretization bias in the forward process.

Posterior-count identity and transition score. The key latent quantity is the posterior expected count. Given both the clean endpoint Y and the noised endpoint R_t , the code computes

$$\bar{\eta}_t(r, y) := \mathbb{E}[K_t \mid R_t = r, Y = y].$$

Once $\bar{\eta}_t(r, y)$ is known, the exact transition score is recovered analytically:

$$\partial_r \log q_t(r \mid y) = \frac{\alpha - 1 + \bar{\eta}_t(r, y)}{r} - \frac{1}{s_t}.$$

Equivalently, the relative score satisfies

$$1 + r \partial_r \log q_t(r \mid y) = \alpha + \bar{\eta}_t(r, y) - \frac{r}{s_t}.$$

So even at the transition level, the score depends on the latent count only through its posterior expectation.

Regression on posterior counts. At training time, the clean state X_0 is known because it comes from the dataset. So the benchmark code can compute exact labels

$$\bar{\eta}_t(X_t, X_0) = \mathbb{E}[K_t \mid X_t, X_0].$$

However, the neural network does *not* receive X_0 as input. It only receives (X_t, t, Z) , where Z is task-specific conditioning such as Sudoku clues or a DNA signal track. This is where the L^2 projection argument enters.

Define the population objective

$$\mathcal{J}(g) = \mathbb{E} \left[\left(\bar{\eta}_t(X_t, X_0) - g(X_t, t, Z) \right)^2 \right].$$

By the geometry of conditional expectation, the minimizer over square-integrable functions of (X_t, t, Z) is

$$g^*(x, t, Z) = \mathbb{E}[\bar{\eta}_t(X_t, X_0) \mid X_t = x, t, Z].$$

Using the tower property,

$$g^*(x, t, Z) = \mathbb{E}[\mathbb{E}[K_t \mid X_t, X_0] \mid X_t = x, t, Z] = \mathbb{E}[K_t \mid X_t = x, t, Z].$$

So training on the exact count label $\bar{\eta}_t(X_t, X_0)$ is not merely convenient; it is an unbiased regression route to the posterior count function the model actually needs at generation time.

Neural parameterization and analytic score recovery. In the local implementation, the CIR model is trained in stabilized log-count space. The network predicts

$$g_\theta(x_t, t, Z) \approx \log(1 + \eta_t(x_t, Z)), \quad \hat{\eta}_\theta(x_t, t, Z) = \exp(g_\theta(x_t, t, Z)) - 1,$$

and the empirical loss is

$$\hat{\mathcal{L}}(\theta) = \frac{1}{n} \sum_{i=1}^n \left(\bar{\eta}_{t_i}(X_{t_i}^{(i)}, X_0^{(i)}) - \hat{\eta}_\theta(X_{t_i}^{(i)}, t_i, Z_i) \right)^2.$$

This matches the actual benchmark implementation, which fits $\log(1 + \text{posterior count})$ rather than raw counts.

After predicting $\hat{\eta}_\theta$, the score is recovered analytically coordinatewise:

$$\hat{\rho}_t(x_t, Z) = \frac{\alpha - 1 + \hat{\eta}_\theta(x_t, t, Z)}{x_t} - \frac{1}{s_t}.$$

The implementation also uses the weighted score

$$\hat{w}_t(x_t, Z) := \sigma^2 x_t \hat{\rho}_t(x_t, Z) = \sigma^2 (\alpha - 1 + \hat{\eta}_\theta(x_t, t, Z)) - \sigma^2 \frac{x_t}{s_t},$$

because this is the stable reverse-drift correction consumed by the reverse samplers.

Small-time weighting and reverse sampling. The small-time part is the numerically stiff part of the problem. Since

$$\frac{1}{s_t} = \frac{2\kappa}{\sigma^2(1 - e^{-\kappa t})} \sim \frac{1}{t} \quad \text{as } t \downarrow 0,$$

both the score and the latent posterior-count target become sharper near the clean-data end of the process. The code therefore oversamples small diffusion times through importance sampling. If $p(t)$ is the nominal time law and $q(t)$ is a proposal that places more mass near zero, then the regression objective can be written as

$$\mathcal{L}(\theta) = \mathbb{E}_{t \sim q} \left[\frac{p(t)}{q(t)} \mathbb{E} \left[(\bar{\eta}_t(X_t, X_0) - \hat{\eta}_\theta(X_t, t, Z))^2 \mid t \right] \right].$$

After training, the reverse process is run in orthant coordinates and normalized back to the product simplex only at the end. The CIR branch tunes among reverse SDE, reverse ODE, and Heun-type ODE samplers, while the baseline branch uses a log-space multiplicative reverse update on the simplex. In both cases the tune split is used only for sampler selection; the final paper tables report held-out test performance.

4 Computational Experiments

In both benchmarks, the neural network is used to approximate the CIR score function indirectly rather than directly. The model receives a noised example x_t , the diffusion time t , and task-specific conditioning information. For the CIR method, the network learns the posterior-count map

$$g_\theta(x_t, t, Z) \approx \log(1 + \eta_t(x_t, Z)),$$

and the full score is then recovered analytically from $\hat{\eta}_\theta$. This is the main computational message of the paper: we only need to learn the posterior expected latent count, not the whole score field from scratch. The comparison method is a matched direct Dirichlet score baseline with the same seed set $\{0, 1, 2\}$, the same backbone family within each task, and the same tune/test protocol. In both branches, the torch model keeps a small internal validation split for early stopping, while the explicit tune split is reserved for reverse-sampler selection.

Across both tasks, the matched comparison is the following. The CIR branch uses exact CIR noising, trains on posterior expected latent counts, and recovers the score analytically. The baseline branch uses product-Dirichlet noising, trains on Dirichlet concentration targets, and recovers the baseline weighted score as concentration minus one. Within each task, the two methods share the same train/tune/test split, the same seed set $\{0, 1, 2\}$, the same backbone family, and the same tune-then-test evaluation flow.

All reported runs were launched as single-GPU Slurm jobs on the shared `gpu-v100` partition. The recorded runtime metadata show one Tesla V100 GPU per run. The DDSM promoter runs used Tesla V100-PCIE-16GB devices, while the Sudoku runs used Tesla V100-PCIE-16GB and Tesla V100-PCIE-32GB devices. Across both tasks, small-time MSE is computed on the earliest 20% of sampled diffusion times. Since both pipelines sample diffusion times on $[0.2, 1.8]$, the reported small-time region corresponds to test examples with $t \leq 0.52$. We also report unstable-step fraction from the reverse sampler diagnostics: this is the fraction of reverse steps whose proposals were nonfinite or had to be repaired numerically.

Method	Missing Acc. $\uparrow$	Board Validity $\uparrow$	Exact Solve $\uparrow$	Total Time (min) $\downarrow$	Small- t MSE $\downarrow$	Instability $\downarrow$
CIR Tweedie	0.747 $\pm$ 0.003	0.412 $\pm$ 0.005	0.412 $\pm$ 0.005	58.6 $\pm$ 1.6	0.067 $\pm$ 0.012	0.814 $\pm$ 0.264
Dirichlet Score	0.741 $\pm$ 0.019	0.371 $\pm$ 0.024	0.371 $\pm$ 0.024	65.6 $\pm$ 5.8	1.317 $\pm$ 0.116	1.000 $\pm$ 0.000

Table 1: Matched three-seed Sudoku completion benchmark under the shared torch product-simplex backbone family and decoder budget.

4.1 Sudoku Benchmark

We discuss Sudoku first because it is the smaller and more controlled benchmark in the paper. The Sudoku benchmark is generated entirely inside the local pipeline. For each seed we generate 900 solved boards by applying random Sudoku symmetries to a canonical valid board, sample clue counts from $\{30, 40\}$, and form a stratified 0.7/0.15/0.15 train/tune/test split. Training boards need not have unique solutions, but tune and test boards are filtered to have unique solutions through a bounded backtracking-based construction routine. The CIR and Dirichlet models use the same transformer-style product-simplex backbone with model dimension 128, 4 attention heads, 4 layers, batch size 64, learning rate 10^{-3} , and a maximum of 50 epochs.

The conditioning payload for Sudoku is richer than a plain masked board. The network sees the puzzle clue values, a clue mask indicating fixed cells, and extra block features that encode candidate digits together with row, column, and box position indicators. The final decoder budget is matched across methods: both use the same repair decoder, which first builds a row-valid board from the final cellwise probabilities and then applies within-row swaps to reduce column and box conflicts. The decoder runs up to 200 local-repair steps and uses 2 restarts, meaning one direct repair attempt and one perturbed restart to escape bad local minima.

Sampler selection is also explicit. On the tune split, the CIR branch compares three candidates: a 40-step reverse SDE, a 60-step reverse SDE, and a 50-step Heun-type ODE. The Dirichlet branch compares three log-space multiplicative samplers with 40, 60, and 80 steps. To keep tune-time cost bounded, the Sudoku code evaluates these candidates on a random subset of up to 64 tune boards. Candidates are ranked lexicographically by full-board validity, exact-solution rate, missing-cell accuracy, invalid-constraint count, and finally unstable-step fraction.

The saved runs show the clearest win for the CIR target in this paper. Table 1 reports missing-cell accuracy of 0.747 ± 0.003 for CIR versus 0.741 ± 0.019 for the direct Dirichlet score baseline. Here missing-cell accuracy means the fraction of puzzle-hidden cells filled with the correct digit. Board validity improves from 0.371 ± 0.024 to 0.412 ± 0.005 , where board validity means the fraction of completed boards satisfying every row, column, and 3×3 box constraint. Small-time MSE drops sharply from 1.317 ± 0.116 to 0.067 ± 0.012 , and total runtime drops from 65.6 ± 5.8 minutes to 58.6 ± 1.6 minutes. Sampler instability is also lower for CIR (0.814 ± 0.264 versus 1.000 ± 0.000).

Note that full-board validity and exact-solution rate are identical. This happens because the benchmark uses unique-solution puzzles on tune and test, and the decoder keeps the clue cells fixed. Under those conditions, a valid completed board is also the exact held-out solution. So the clearest Sudoku message is not that we found two separate wins, but that the same better score target leads to better missing-cell recovery, more valid completed boards, and faster total runtime under the same decoder budget.

4.2 DNA Promoter Benchmark

We then turn to the harder DNA promoter benchmark. The DNA benchmark follows the official DDSM promoter_design release. The split is chromosome based: chr8 and chr9 are the test split, chr10 is the tune split, and the remaining chromosomes are used for training. Each example is a 1024bp promoter window from the human reference genome together with a strand-aligned CAGE signal track used as conditioning. With the current cap of 40,000 TSS rows, each seed yields 35,523 training sequences, 1,562 tune sequences, and 2,915 test sequences. On the training split only, the local loader also applies a random promoter offset of up to ± 100 bp around the released TSS location; validation and test use zero offset.

Both methods again use the same transformer-style product-simplex backbone, now with model dimension 128, 4 attention heads, 4 layers, dropout 0.1, batch size 4, and a maximum of 40 epochs.

Method	SP-MSE ↓	Pos.-Freq. ℓ_1 ↓	Total Time (min) ↓	Small- t MSE ↓	Score MSE ↓	Instability ↓
CIR Tweedie	0.117 ± 0.006	0.056 ± 0.009	99.6 ± 0.2	0.047 ± 0.001	0.061 ± 0.000	1.000 ± 0.000
Dirichlet Score	0.116 ± 0.001	0.051 ± 0.003	71.7 ± 19.4	0.870 ± 0.546	0.416 ± 0.225	0.365 ± 0.044

Table 2: Matched three-seed benchmark on the official DDSM promoter-design task under the shared chromosome split and SEI-based evaluation.

Evaluation uses a frozen pretrained SEI model loaded from the DDSM assets rather than retrained inside this project. In the current saved runs, the selected SEI feature is H3K4me3. The headline application metric, SP-MSE, is the mean squared error between the SEI-predicted target activity of the generated sequence and that of the held-out real sequence on this target feature.

The CIR branch tunes three candidates on the full tune split: a 40-step reverse SDE, a 60-step reverse SDE, and a 50-step Heun-type ODE. The Dirichlet branch tunes three multiplicative simplex samplers with 40, 60, and 80 steps. DDSM candidates are ranked lexicographically by tune-split SP-MSE and then unstable-step fraction, because the application metric is the primary benchmark objective.

Table 2 shows that SP-MSE is essentially tied: 0.117 ± 0.006 for CIR and 0.116 ± 0.001 for the direct Dirichlet score baseline. Position-frequency ℓ_1 is also slightly better for the baseline. However, the score diagnostics strongly favor CIR. Small-time MSE drops from 0.870 ± 0.546 to 0.047 ± 0.001 , and score MSE drops from 0.416 ± 0.225 to 0.061 ± 0.000 . These are exactly the metrics on which posterior-count regression should help most.

The drawback is that the cleaner learned score does not yet produce a cleaner end-to-end generator. The current CIR sampler is slower overall (99.6 ± 0.2 minutes versus 71.7 ± 19.4 minutes) and less stable (1.000 ± 0.000 versus 0.365 ± 0.044 unstable-step fraction). The runtime breakdown shows that most of this gap comes from the fitting stage rather than the tune or generation stage: CIR spends about 86.4 minutes fitting and 9.9 minutes tuning the sampler, while the baseline spends about 57.8 minutes fitting and 9.2 minutes tuning. So the DNA conclusion is narrower than the Sudoku conclusion: the CIR Tweedie/count target is easier to learn in the sense of score diagnostics, but stronger reverse-time sampling is still needed before that advantage becomes a downstream win on the promoter-design task.

5 Conclusion

The main outcome of the project is a simpler training framework for simplex-aware diffusion. Instead of learning the entire CIR score directly, we learn a posterior expected latent count with a neural network and recover the score analytically. That turns score learning into a standard regression problem while keeping exact CIR forward simulation in the loop.

The saved benchmark results support that design. On Sudoku, the CIR Tweedie/count target improves the main task metrics, improves the score diagnostics, and reduces total runtime under the same decoder budget. On the official DDSM DNA benchmark, the same target produces a much cleaner learned score but does not yet improve the headline SEI-based application metric or end-to-end stability. So the most credible next step is to develop a stronger reverse-time sampling layer that can convert the better score surrogate into better downstream generation.

References

- [AS17] Benjamin Arras and Yvik Swan. A stroll along the gamma. *Stochastic Processes and their Applications*, 127(11):3661–3688, 2017.
- [AST⁺23] Pavel Avdeyev, Chenlai Shi, Yuhao Tan, Kseniia Dudnyk, and Jian Zhou. Dirichlet diffusion score model for biological sequence generation. In *International Conference on Machine Learning*, pages 1276–1301. PMLR, 2023.
- [RDD22] Pierre H Richemond, Sander Dieleman, and Arnaud Doucet. Categorical sdes with simplex diffusion. *arXiv preprint arXiv:2210.14784*, 2022.

[SSDK⁺21] Yang Song, Jascha Sohl-Dickstein, Diederik P. Kingma, Abhishek Kumar, Stefano Ermon, and Ben Poole. Score-based generative modeling through stochastic differential equations. In *International Conference on Learning Representations*, 2021.